Programação Avançada

Unidade 1: POO (Programação Orientada a Objetos)

Aula 04: Programação Orientada a Objetos - Encapsulamento

Prof. Eduardo Corrêa Gonçalves

18/03/2026

Sumário

Introdução

O que é Encapsulamento?

Implementando o Encapsulamento

Modificadores de Acesso

Getters e Setters

Slots

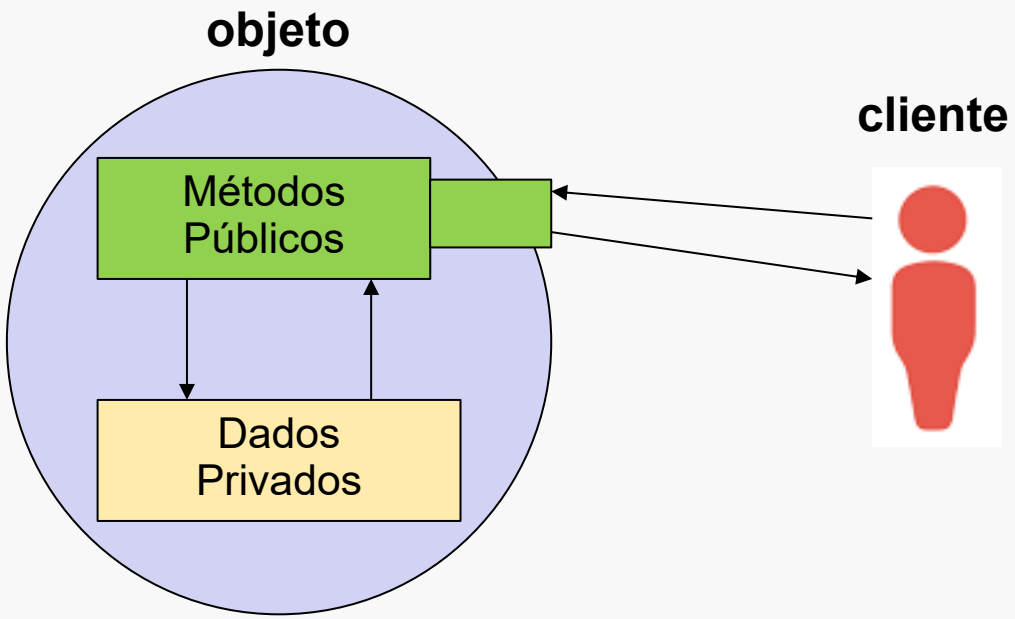
Introdução (1/7)

- **O que é Encapsulamento?***
  - Trata-se de um mecanismo utilizado para lidar com o aumento da complexidade de um sistema.
    - **Ideia básica**:
      - Apresentar para o cliente (usuário) de uma classe **o que** pode ser feito, **sem informar como** é feito.
    - **Benefícios esperados**:
      - **Proteger** os dados manipulados.
      - Obter uma melhor **divisão de responsabilidades** entre as classes.
        - Manter junto em uma classe o que é correlato e separar o que é distinto.

* Definição retirada de Murta, 2013

Introdução (2/7)

- **Níveis de Encapsulamento**
  - Existem diferentes níveis de encapsulamento - do 0 ao 4.
  - Um dos mais utilizados é o **nível 2**, onde:
    - Clientes não tem acesso direto aos dados (atributos da classe).
    - Eles sempre interagem com os objetos através dos métodos.



- ***Obs.**: em nosso curso, nem sempre trabalharemos no nível 2 (seremos menos restritivos...)*

Introdução (3/7)

- **Exemplos de Encapsulamento Nível 2**
  - **Exemplo 1 – set**
    - Para usar a classe **set** (conjunto) um usuário **não precisa** saber que ela foi implementada com um *bucket array* e nem precisa acessar diretamente esta estrutura.
    - Mas precisa saber manipular os objetos da classe através dos métodos add(), remove(), update(), etc.

```
>>> c1 = {"John", "Ringo", "Richards"}
>>> lst = ["Stu", "Pete"]
>>> c1.add("Paul")
>>> c1
{'Richards', 'John', 'Ringo', 'Paul'}

>>> c1.add("George")
>>> c1.remove("Richards")
>>> c1.update(lst)
>>> c1
{'George', 'Ringo', 'Paul', 'Stu', 'John', 'Pete'}
```

Introdução (4/7)

- **Exemplos de Encapsulamento Nível 2**
  - **Exemplo 2**: Para utilizar o classificador **Naive Bayes Categórico** (CategoricalNB) da biblioteca scikit-learn, um usuário não precisa saber detalhes sobre a implementação.
    - Mas precisa saber usar os métodos fit() para treinar o modelo e predict() para classificar um objeto.

```
import numpy as np          # PARTE 1: gera uma base de dados
rng = np.random.RandomState(1)
X = rng.randint(5, size=(6, 100))
Y = np.array([1, 2, 3, 4, 5, 6])

#PARTE 2: classificação
from sklearn.naive_bayes import CategoricalNB
clf = CategoricalNB()       # instancia um objeto da classe CategoricalNB
clf.fit(X, Y)               # executa o método fit() para treinar o modelo
print(clf.predict(X[2:3]))  # executa o método predict() para classificar

[3]
```

Exemplo retirado de: https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.CategoricalNB.html#sklearn.naive_bayes.CategoricalNB

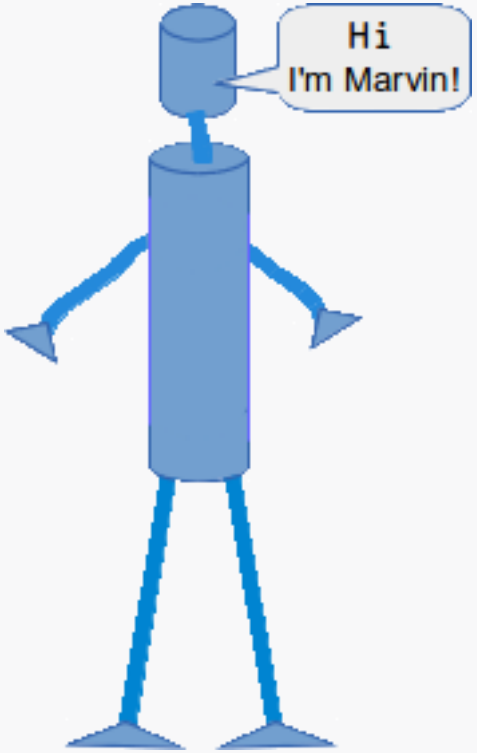
Introdução (5/7)

- **Congeneridade***
  - O encapsulamento é uma das ferramentas que a POO utiliza para **ajudar a reduzir a congeneridade** total de um sistema.
  - A congeneridade entre dois elementos A e B de um sistema significa que:
    - Se A for modificado, B terá que ser modificado ou ao menos verificado.
    - Pode ocorrer uma modificação no sistema que obrigue modificações conjuntas em A e B.
  - Logo, ao minimizarmos a congeneridade, **facilitamos a manutenção** do sistema.
    - Reduzimos o número de alterações necessárias no código quando for preciso implementar uma nova funcionalidade ou alterar uma já existente.

* Definição retirada de Murta, 2013

Introdução (6/7)

- **Encapsulamento no Python**
  - Nesta aula, veremos **três** abordagens para encapsulamento que podem ser utilizadas no Python:
    - Modificadores de acesso
    - Getters e Setters
    - Slots
  - Os exemplos utilizarão a classe Robot, introduzida no laboratório da última aula*.



- Originalmente apresentada em: https://www.python-course.eu/python3_object_oriented_programming.php

Introdução (7/7)

- **A classe Robot**
  - Duas propriedades: nome e ano_fabricacao
  - Um método: falar_oi()

robot.py

```python
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if (ano_fabr is None) or (type(ano_fabr) != int) or (ano_fabr > 2024):
            self.ano_fabricacao = 2024
        else:
            self.ano_fabricacao = ano_fabr


    def falar_oi(self):
        print(f'oi sou {self.nome} de {self.ano_fabricacao}')

```

Modificadores de Acesso (1/8)

- **A Classe Robot** : Veja que o código no construtor impede um ano de fabricação inválido ou nulo (nesses casos, atribui 2024).

robot.py

```python
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if (ano_fabr is None) or (type(ano_fabr) != int) or (ano_fabr > 2024):
            self.ano_fabricacao = 2024
        else:
            self.ano_fabricacao = ano_fabr


    def falar_oi(self):
        print(f'oi sou {self.nome} de {self.ano_fabricacao}')

```

Shell

```
>>> %Run robot.py
>>> x = Robot("Zed")  # ano ausente -> joga 2024
>>> x.falar_oi()
 oi sou Zed de 2024
>>> y = Robot("Gort", "bla bla") # ano errado -> joga 2024
>>> y.falar_oi()
 oi sou Gort de 2024
>>> z = Robot("Hal", 1968) # ano válido !
>>> z.falar_oi()
 oi sou Hal de 1968
>>> # veja que o ano é validado no construtor!
>>>
```

Modificadores de Acesso (2/8)

- **Problema**: Depois que o objeto é instanciado, o ano e o nome podem ser mudados a qualquer momento.
  - E assim, o ano poderá ficar com valor inválido, já que o único teste está no construtor.

```
>>> z = Robot("Torg",2008)
>>> z.ano_fabricacao
2008
>>> z.falar_oi()
oi, sou Torg de 2008
>>>
>>> z.ano_fabricacao = "427 A.C."   # atribuído um ano inválido!
>>> z.falar_oi()
oi, sou Torg de 427 A.C.
>>>
```

Modificadores de Acesso (3/8)

- **Solução**: modificar o acesso aos atributos nome e ano_fabricacao, tornando-os **privados**.
  - Em Python, inserimos dois dois underscores ('__') ao atributo para tornar um atributo privado:

```
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.__nome = nome
        if (ano_fabr is None) or (type(ano_fabr) != int) or (ano_fabr > 2024):
            self.__ano_fabricacao = 2024
        else:
            self.__ano_fabricacao = ano_fabr


    def falar_oi(self):
        print(f'oi sou {self.__nome} de {self.__ano_fabricacao}')
```

  - Agora os atributos se chamam:
    - __nome
    - __ano_fabricacao

Modificadores de Acesso (4/8)

- Agora “ano_fabricacao” e “nome” não existem mais:

```
>>> z = Robot("Torg",2008)
>>> z.falar_oi()
oi, sou Torg de 2008
>>>
>>> z.ano_fabricacao
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
AttributeError: 'Robot' object has no attribute 'ano_fabricacao'
```

Modificadores de Acesso (5/8)

- E se eu tentar mudar o valor do ano usando “.ano_fabricacao” ou “.__ano_fabricacao”, não vou alterar o atributo.

```
>>> z.ano_fabricacao = "XXXXX"
>>> z.__ano_fabricacao = "AAAAAA"
>>> z.falar_oi()
oi, sou Torg de 2008
```

Modificadores de Acesso (6/8)

- Curiosamente, o Python não tornou o atributo privado de verdade, mas aplicou um “truque”:
  - Ao colocar o prefixo __ no atributo, o Python apenas renomeia '__nome_do_atributo' para '_nomeClasse__nomeAtributo’
  - Qualquer pessoa que saiba que os atributos privados não são realmente privados, mas "desconfigurados", pode ler e atribuir um valor ao atributo "privado" diretamente.

```
>>> z = Robot("Torg",2008)
>>> z.__dict__
{'_Robot__nome': 'Torg', '_Robot__ano_fabricacao': 2008}
>>>
>>> z._Robot__ano_fabricacao = "abc"
>>> z.falar_oi()
oi, sou Torg de abc
```

  - Essa característica difere o Python de linguagens como C++ e Java, onde o atributo privado é realmente inacessível.

Modificadores de Acesso (7/8)

- De qualquer forma, o uso de dois underscores '__' ao menos serve para **sinalizar** "ao mundo exterior" que o atributo é **privado**.

- De fato, muitos programadores Python não gostam dessa sintaxe e preferem apenas utilizar um underscore '_' para **sinalizar** que o atributo é **protegido**.
  - Nesse caso, o Python nada faz (não aplica nenhum truque para renomear o atributo).
  - A ideia é apenas o desenvolvedor da classe indicar aos clientes que acessar diretamente um atributo protegido pode ser perigoso.

- Figuras retiradas de: https://www.python-course.eu/python3_object_oriented_programming.php

Modificadores de Acesso (8/8)

- Como muitos livros de Estruturas de Dados adotam a sintaxe com um underscore "_" (atributo protegido), adotaremos este modelo na maior parte dos exemplos dessa aula.

```
1  class Robot:
2
3      def __init__(self, nome, ano_fabr=None):
4          self._nome = nome
5          if (ano_fabr is None) or (type(ano_fabr) != int) or (ano_fabr > 2024):
6              self._ano_fabricacao = 2024
7          else:
8              self._ano_fabricacao = ano_fabr
9
10
11     def falar_oi(self):
12         print(f'oi sou {self._nome} de {self._ano_fabricacao}')
```

Getters e Setters (1/7)

- Na maioria das linguagens orientadas a objetos, uma **maneira controlada** de permitir acesso aos atributos (já que eles são ‘protegidos’) é criar **dois métodos**:
  - **getter**: método que retorna o valor de um atributo privado/protegido de uma classe.
    - Ex: “get_ano_fabricacao()”, “get_nome()”
  - **setter**: método que atribui o valor de um atributo privado/protegido de uma classe.
    - Ex: “set_ano_fabricacao(ano)”, “set_nome(nome)”
- No slide a seguir, veja como fica a classe Robot com o getters e setters.

Getters e Setters (2/7)

- Uma vantagem é que podemos validar as entradas nos setters.

robot_getter_setter.py

```python
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self._nome = nome
        if (ano_fabr is None) or (type(ano_fabr) != int) or (ano_fabr > 2024):
            self._ano_fabricacao = 2024
        else:
            self._ano_fabricacao = ano_fabr

    def falar_oi(self):
        print(f'oi sou {self._nome} de {self._ano_fabricacao}')

    def get_nome(self):
        return self._nome

    def set_nome(self, nome):
        if type(nome) == str: self._nome = nome
        else: print('Nome inválido! Deve ser uma string')

    def get_ano_fabricacao(self):
        return self._ano_fabricacao

    def set_ano_fabricacao(self, ano_fabr):
        if (type(ano_fabr) == int) and (ano_fabr <= 2024):
            self._ano_fabricacao = ano_fabr
        else: print('Ano inválido! Deve ser um inteiro menor que 2024')
```

Getters e Setters (3/7)

- Uma vantagem é que podemos validar as entradas nos setters:

```
>>> x = Robot('ZipZop',2017)
>>> x.falar_oi()
oi, sou ZipZop de 2017
>>>
>>> x.set_nome(1234)
Nome inválido! Deve ser uma string
>>> x.set_ano_fabricacao("bla bla")
Ano inválido! Deve ser um inteiro menor que 2024
>>>
```

- A parte chata é que sou obrigado a escrever o nome do método sempre que quero alterar o valor do atributo.

Getters e Setters (4/7)

- Para resolver essa questão o Python oferece uma oferece uma solução chamada de **properties**.
- Com ela, podemos manter nossos atributos protegidos e implementamos os métodos com um **decorador** chamado **property**.
  - Um decorador (*decorator*) é usado para estender o comportamento de uma função sem que seja preciso modifica-la explicitamente.
  - Além disso, decoradores costumam fornecer uma sintaxe mais simples para acessar uma função (*Syntactic Sugar*).
- Detalhes sobre decoradores estão fora do escopo dessa disciplina (é um tema relacionado à Padrões de Projeto). Basta saber que em Python:
  - Usamos o decorador **@property** para decorar o getter.
  - E o decorador **@setter** para decorar o setter.
- Veja no slide a seguir como fica a classe Robot “decorada”:

Getters e Setters (5/7)

- Observe que os nomes dos métodos agora são simples.

```
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self._nome = nome
        if (ano_fabr is None) or (type(ano_fabr) != int) or (ano_fabr > 2024):
            self._ano_fabricacao = 2024
        else: self._ano_fabricacao = ano_fabr

    def falar_oi(self):
        print(f'oi sou {self._nome} de {self._ano_fabricacao}')

    @property
    def nome(self):
        return self._nome

    @nome.setter
    def nome(self, nome):
        if type(nome) == str: self._nome = nome
        else: print('Nome inválido! Deve ser uma string')

    @property
    def ano_fabricacao(self):
        return self._ano_fabricacao

    @ano_fabricacao.setter
    def ano_fabricacao(self, ano_fabr):
        if (type(ano_fabr) == int) and (ano_fabr <= 2024):
            self._ano_fabricacao = ano_fabr
        else: print('Ano inválido! Deve ser um inteiro menor que 2024')
```

Getters e Setters (6/7)

- Podemos agora chamar os métodos sem os parênteses, como se fossem atributos públicos.

- É uma forma mais elegante de encapsular.

```
>>> h = Robot("Hal",1970)
>>> h.falar_oi()
oi, sou Hal de 1970
>>>
>>> h.ano_fabricacao = 'xxxx'
Ano inválido! Deve ser um inteiro menor que 2024
>>> h.ano_fabricacao = 1968
>>> h.falar_oi()
oi, sou Hal de 1968
```

Getters e Setters (7/7)

- Mas, não esqueça que como no Python, os atributos protegidos são só “sinalizadores”, permanecendo assim a possibilidade de atribuir valores por fora do setter:

```
>>> h = Robot("Hal",1968)
>>> h.falar_oi()
oi, sou Hal de 1968
>>>
>>> h._ano_fabricacao = 2001
>>> h.falar_oi()
oi, sou Hal de 2001
>>>
```

- De qualquer forma, não deixa de ser encapsulamento, pois o **nome real** do atributo é “**_ano_fabricacao**”, mas o nome apresentado ao mundo exterior é “**ano_fabricacao**”.

Slots (1/4)

- Como Python é uma linguagem dinâmica, podemos criar novos atributos para um objeto instanciado a qualquer momento.
- Com isso, nossos objetos ficam **sujeitos à modificações em sua estrutura**, o que pode gerar problemas.

```
>>> z = Robot("Torg",2008)
>>> z.__dict__
{'_nome': 'Torg', '_ano_fabricacao': 2008}
>>>
>>> z.melhor_amigo = 'Hal'
>>> z.lata_velha = True
>>> z.preco = 99.99
>>> z.__dict__
{'_nome': 'Torg', '_ano_fabricacao': 2008, 'melhor_amigo': 'Hal', 'lata_velha':
True, 'preco': 99.99}
>>>
```

Slots (2/4)

- Para evitar isso podemos utilizar uma variável embutida no Python chamada **__slots__**
- Essa variável pode guardar uma lista fixa, com os atributos da classe

```
class Robot:

    __slots__ = ['_nome', '_ano_fabricacao']

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if (ano_fabr==None) or (type(ano_fabr) != int) or (ano_fabr > 2020):
```

Slots (3/4)

- Agora, se tentarmos adicionar qualquer atributo ocorrerá um erro.

```
>>> z.preco = 99.99
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
AttributeError: 'Robot' object has no attribute 'preco'
>>>
```

Slots (4/4)

- Ao atribuir um valor para __slots__ , o do Python vai entender que queremos **excluir o __dict__** da classe.

- Assim, não há mais possibilidade de adicionar atributos ao dicionário da classe que é responsável por armazenar atributos de instância.

```
>>> >>> z.__dict__
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
AttributeError: 'Robot' object has no attribute '__dict__'
>>>
>>> z.__slots__
['_nome', '_ano_fabricacao']
```

Considerações Finais (1/2)

- **Encapsulamento**
  - Princípio usado na POO que consiste em apresentar para o cliente (usuário) de uma classe **o que** pode ser feito, **sem informar como** é feito.
  - O encapsulamento ajuda a reduzir a congeneridade do sistema, tornando mais fácil a sua manutenção.

Considerações Finais (2/2)

- **Como saber que técnicas de encapsulamento devo utilizar?**
  - Depende de vários fatores:
    - Normas da empresa onde você trabalha.
    - Importância e tamanho de seu programa.
    - Componentes do sistema que você deseja proteger mais.
    - ...
  - Uma regra geral é o bom senso... Normalmente não precisamos "radicalizar muito".
    - Por exemplo, nem sempre é necessário criar getters e setters para todos os atributos da classe.
    - Na disciplina Programação Avançada, adotaremos no máximo o uso de atributos protegidos (nome com um sublinhado "_").

Tarefa

- **(1)** Defina uma classe chamada **Flor**, com três atributos privados: nome (nome da flor, string), petalas (número de pétalas, int) e preco (preço da flor, float).
  - A classe deverá incluir um método construtor para inicializar as variáveis e métodos get e set básicos (sem decoradores) para, respectivamente, recuperar e atribuir valores para estes atributos.
  - Tanto o construtor como os setters devem validar os tipos de cada dado de entrada e imprimir uma mensagem de erro caso o tipo errado seja especificado. Além disso, se o tipo errado for especificado, o valor None deverá ser atribuído ao atributo.
  - Depois de definir a classe, instancie um objeto “f”, da classe Flor, com as seguintes características: “margarida”, 13 pétalas, R$ 85,90. Imprima o valor dos três atributos usando os getters.
  - Por fim, utilize os setters adequados para alterar o número de pétalas para 21 e o preço para R$ 99,90. Imprima o valor dos três atributos usando os getters.

- **(2)** Crie uma classe chamada **FlorNova** com a mesma definição da classe Flor, do exercício anterior, mas agora utilizando decoradores nos getters e setters.

Referências

- Python 3 Tutorial – Object Oriented Programming. https://www.python-course.eu/python3_object_oriented_programming.php

- CAELUM (2020). Python e Orientação a Objetos – Curso PY-14. (*Cap. 8*).

- Murta, L. (2013). Princípios de POO. http://www2.ic.uff.br/~anselmo/cursos/TPA/apresentacoes/POO.pdf

- Hjelle G. A. (2018). Primer on Python Decorators. https://realpython.com/primer-on-python-decorators/

- Shaik, H. K. (2018). Property vs. Getters and Setters in Python. https://www.datacamp.com/community/tutorials/property-getters-setters